

# OzoneTrckr — Firmware Flow

How a measurement run executes, from cold boot through the loop to deep sleep — and the three independent ways the data gets off the device.

**SETUP()** runs once on power-up / reset

## Wake the hardware safely

The board boots and powers up the analog front-end in the right order — analog off, then a cold start — before anything uses the SPI bus the ADC and screen share, so the ADC never desyncs. `setup()`

## Turn on the screen, centre the filter

The TFT lights up (black at first) and the filter servo eases to its middle position. `displayInit()`

## Find the sun

The four light sensors steer the two mount servos until all quadrants are equally bright — i.e. pointed at the sun. `sunflowerFindSun()`

TRACKER

## Start the clock chip

Bring up the battery-backed real-time clock on the I<sup>2</sup>C bus; it holds the time even between runs. `rtc.begin()`

## DATA RESILIENCE — 3 CHANNELS

- F Flash** — every block, to `/lastrun.csv`. Survives WiFi loss & battery death.
- S Serial** — live [...] rows during the run, full CSV at end + next boot.
- W WiFi** — TCP upload to the PC's `ncat -l 5000`.

## Mount the flash storage

FLASH

Open the on-chip filesystem *without* auto-formatting, so a damaged-but-readable previous run still gets a chance to be rescued below. `storageBegin()`

TFT Flash backup ready

## Join WiFi and announce the device

WIFI

Connect with a fixed address, advertise the name `ozone.local`, and open a command port so a run can be ended remotely. `commandLinkBegin()`

TFT IP 192.168.1.50

## Set the exact time

NTP

Ask an internet time server for the precise UTC and update the clock; if that fails, just trust the clock chip. `ntpSyncUTC()`

TFT Clock: NTP synced

## Rescue the previous run

SERIAL + WIFI

If an earlier run is still on flash, send it out over serial **and** WiFi before it gets overwritten — so a missed upload never loses data. `uploadLastRun()` · `dumpLastRunToSerial()`

TFT Last run sent (WiFi)

## Check the flash can be written

FLASH

Only now — after rescue — write-test the filesystem and reformat it only if it is genuinely corrupt. `storageEnsureWritable()`

## TFT STATUS DOTS

**W** **WiFi:** green = good, orange **w** = weak, red **x** = down. RSSI shown below.

**F** **Flash:** green = logging & last write OK; red = off / write failed (live, per block).

## COLOUR = SUBSYSTEM

- setup / general
- measurement & tracker
- WiFi / NTP / TCP
- flash storage (FFat)
- serial / delivery

## Bring up the sensors

Read the weather sensor, load the saved site coordinates (the GPS stays asleep), and configure the ADC and its data-ready interrupt. `setup_ADC_CARD()`

## Zero the ADC (dark calibration)

With no light in, measure each channel's offset and noise and load it into the ADC; the values are saved in the file header. `offsetCalibration()`

TFT OFFSET CAL · Ch1 / Ch2

## Open the run file

FLASH

Create the rolling CSV, write the header, and confirm the write landed; from here every block is saved as it is measured.

`runFileBegin()` → `/lastrun.csv`

TFT Logging to flash

## THE RUN

repeats ~every 32 s · 20 readings per loop

### WIFI

#### Keep WiFi alive

Each loop, reconnect at once if the link dropped and nudge it so the router can't quietly kick the device off.

```
wifiKeepalive()
```

### TRACKER

#### Re-aim and work out where the sun is

Every 5th loop the mount re-centres on the sun; every loop the sun's elevation and azimuth are computed from the clock and coordinates. `sunflowerFindSun()` · `sunPosition()`

#### Measure through filter 1

Ease the filter to its first position, then take 10 readings — each one measures both channels, saves a row to flash + serial, and refreshes the live screen.

```
filter_rotation() · measurement() ·  
storeMeasurement()
```

#### Measure through filter 2

Swap to the second filter position and take 10 more the same way. A stop request never cuts a phase short — you always get a full 10 + 10.

TFT 17:42:05 28/06/26 · Ch1 612.34 Ch2 588.10 – the live dashboard, redrawn every reading

↻ repeat the loop...

#### ...until any end condition:

- the planned number of readings is reached, or
- the **BOOT button** is pressed (works with no WiFi), or
- a **stop/dump** command arrives over WiFi.

## END OF RUN

deliver the data, then sleep

### Close the run file

The whole run is already on flash (saved one block at a time), so this just flushes and closes it. `runFileEnd()`

#### ① Serial

Print the CSV to a USB terminal if one is attached — works with no WiFi.

`dumpLastRunToSerial()`

#### ② WiFi → PC

Send the file to the PC's listener; keep retrying for a while (BOOT skips the wait). `uploadLastRun()`

#### ③ Flash (kept)

Stays saved on flash and is re-sent on the next power-up.

TFT Sending data... → Upload done

### Park and power down

Return the filter, put the ADC to sleep, cut analog power, cleanly unmount flash, and hold the servo pins steady so they don't twitch. `storageEnd()`

## SLEEP

### Sleep until the next placement

The board switches fully off with no wake timer; you power-cycle to start the next run — which also re-sends this one if its upload was missed. `esp_deep_sleep_start()`

TFT (display off)

## Setup — setup\_ADC\_CARD()

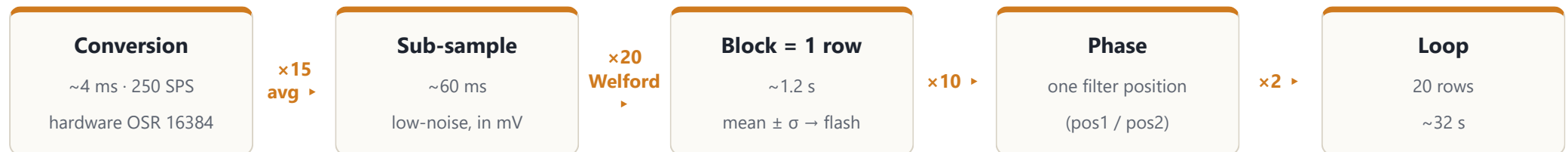
|            |                                                                                  |
|------------|----------------------------------------------------------------------------------|
| Device     | ADS131M04 — 24-bit, 4-ch simultaneous $\Delta\Sigma$ , SPI (bus shared with TFT) |
| Data rate  | ~250 SPS · High-Res mode · OSR 16384 (lowest noise)                              |
| Channels   | CH0 + CH1 stored (CH2 also enabled); differential AINxP/AINxN                    |
| PGA gain   | 1× on CH0 / CH1 / CH2                                                            |
| Full scale | FS = $2^{23}$ = 8 388 608 counts, scaled to PREF = 1200 mV                       |
| Bring-up   | RESET → STANDBY → write CLOCK/CFG/THRSHLD → WAKEUP → one flush read              |

## DRDY handshake — GPIO6, falling-edge IRQ

DRDY drops LOW when a conversion is ready and **stays low until the frame is clocked out over SPI** — it is *not* a free-running pulse in this configuration. The ISR sets a **volatile** flag; the sampling loop sees it, clears it, and calls readADC(), which reads the 24-bit words and releases DRDY for the next conversion.

**So every conversion must be read** — an interrupt-only wait with no read would deadlock. A 2 s no-DRDY timeout is the safety net (it flags "analog board disconnected").

## Sampling hierarchy — how one stored row is built

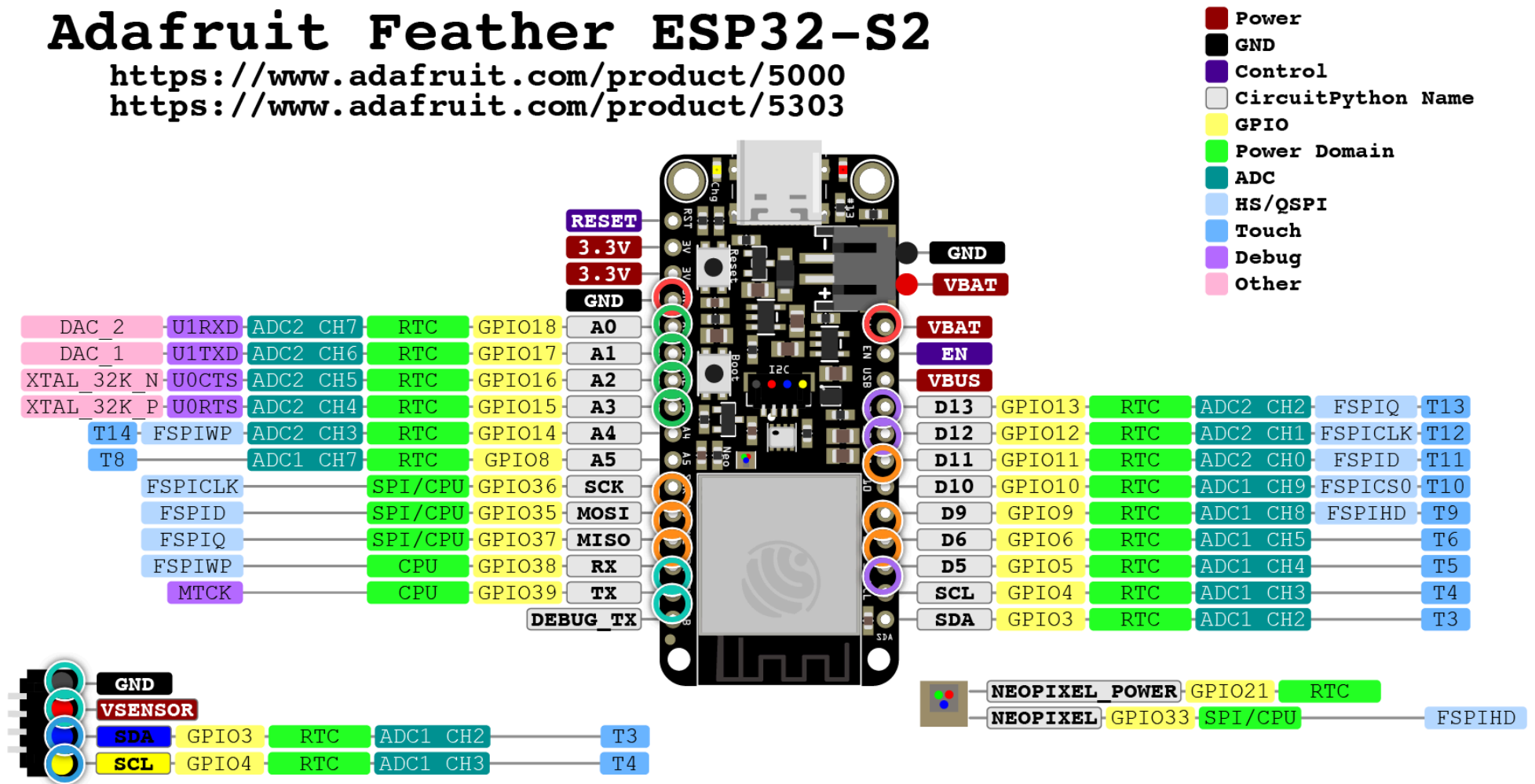


**Counts → mV:**  $V = (\Sigma \text{counts} / n) / \text{FS} \times 1200 \text{ mV}$ , accumulated in double; mean and  $\sigma$  computed with Welford's algorithm for numerical stability. **Settling guard:** after every CMD\_WAKEUP (i.e. each filter move) the first **25 conversions (~100 ms) are discarded** so the wake transient never lands in a block. **Offset cal:** 100 zero-input sub-samples are averaged into the ADC's hardware offset register and reported as mean  $\pm \sigma$  in the CSV header. **Quiet windows:** the ADC is put to STANDBY during every filter-servo move, so servo current spikes and SPI traffic never coincide with sampling.

# Adafruit Feather ESP32-S2

<https://www.adafruit.com/product/5000>

<https://www.adafruit.com/product/5303>



Stemma QT Power pin VSENSOR and I2C pull-up resistors are enabled/disabled by GPIO 7.

If you have a rev B board, set IO7 LOW to enable.

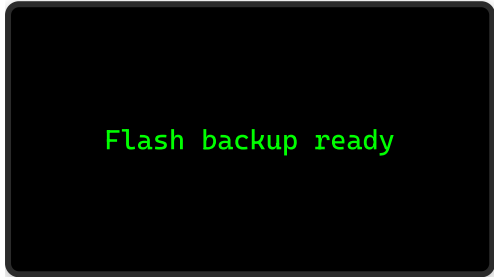
If you have a rev C board, set IO7 HIGH to enable.

| PIN  | GPIO | FUNCTION                                                     | SUBSYSTEM                  |
|------|------|--------------------------------------------------------------|----------------------------|
| SCK  | 36   | ADC SPI clock (shared with TFT)                              | ● ADC board                |
| MOSI | 35   | ADC SPI MOSI                                                 | ● ADC board                |
| MISO | 37   | ADC SPI MISO                                                 | ● ADC board                |
| D11  | 11   | analog-board power enable (EN)                               | ● ADC board                |
| D9   | 9    | ADC chip-select (CS)                                         | ● ADC board                |
| D6   | 6    | ADC data-ready (DRDY, interrupt)                             | ● ADC board                |
| D13  | 13   | tracker servo — vertical (elevation)                         | ● Servos                   |
| D12  | 12   | tracker servo — horizontal (azimuth)                         | ● Servos                   |
| D5   | 5    | filter servo (pos1 / pos2 / cal)                             | ● Servos                   |
| A0   | 18   | LDR bottom-left                                              | ● Sun sensors              |
| A1   | 17   | LDR top-right                                                | ● Sun sensors              |
| A2   | 16   | LDR bottom-right                                             | ● Sun sensors              |
| A3   | 15   | LDR top-left                                                 | ● Sun sensors              |
| SDA  | 3    | I <sup>2</sup> C data — DS3231 RTC + BME280 (via STEMMA QT)  | ● I <sup>2</sup> C sensors |
| SCL  | 4    | I <sup>2</sup> C clock — DS3231 RTC + BME280 (via STEMMA QT) | ● I <sup>2</sup> C sensors |
| RX   | 38   | GPS UART RX (module TX → Feather)                            | ● GPS                      |

| PIN     | GPIO | FUNCTION                                        | SUBSYSTEM |
|---------|------|-------------------------------------------------|-----------|
| TX      | 39   | GPS UART TX (Feather → module RX)               | ● GPS     |
| VSENSOR | —    | GPS + I <sup>2</sup> C-sensor power (STEMMA QT) | ● GPS     |
| VBAT    | —    | analog-board supply (+)                         | ● Power   |
| GND     | —    | analog-board ground                             | ● Power   |

Not a header pin, so not marked above: **GPIO0** = the onboard **BOOT** button (WiFi-free early-stop). The DS3231 RTC and BME280 share the **STEMMA QT** connector (SDA/SCL + VSENSOR/GND); the GPS taps the same connector's **VSENSOR/GND** for power but talks over the header **RX/TX** (GPIO38/39) — it is held in backup/asleep this build, so its UART is idle and coordinates are hardcoded. The **analog board** is powered from **VBAT + GND**; the three servos run off a separate 5 V supply that shares ground with the Feather.

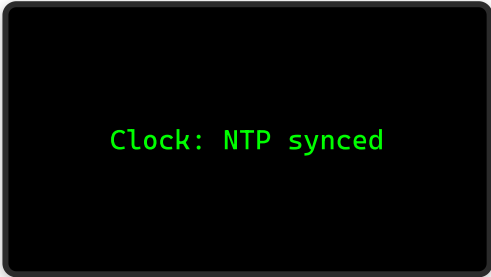
TFT

 what the 240×135 ST7789 shows, in run order


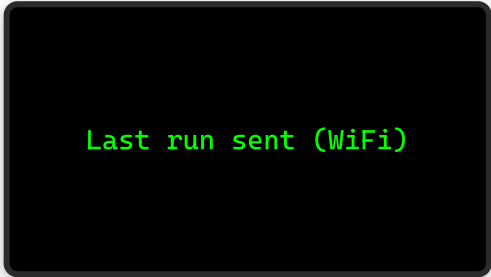
**Setup · flash** — FFat mounted (red FLASH MOUNT FAILED if not).



**Setup · wifi** — device address, no serial needed (also `ozone.local`).



**Setup · clock** — or `Clock: RTC kept /` amber `Clock: build-time`.



**Recovery** — previous run re-delivered over serial + WiFi before it's overwritten.

```
OFFSET CAL
Zero-input offset [mV]

Ch1: 0.1240
Ch2: -0.0310
```

Offset cal — per-channel dark offset, written into the CSV header.

```
17:42:05 28/06/26
N44.5302 E14.4706
EL +47.32 deg
AZ 215.40 SW
Ch1: 612.34 0.87
Ch2: 588.10 0.92
```

Live dashboard — redrawn every block (flicker-free, in-place). Top-right: WiFi dot + RSSI (orange **w** weak / red **x** down), flash-health dot. Elevation is colour-coded by sun height.

```
Sending data ...
```

End of run — then Upload done (green) or On flash + serial; the panel blanks (displayOff()) before deep sleep.

HOST commands you run on the PC — Windows & macOS

| TASK                                                                                 | WIN WINDOWS — POWERSHELL                | MAC MACOS — TERMINAL (ZSH)                                             |
|--------------------------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------|
| Receive a run<br>start the listener first, then end the run                          | <code>ncat -l 5000 &gt; data.csv</code> | <code>nc -l 5000 &gt; data.csv</code>                                  |
| Reach the device                                                                     | <code>ping ozone.local</code>           | <code>ping ozone.local</code><br><code>dns-sd -G v4 ozone.local</code> |
| End run early + send<br>connect, then type <code>send</code> (or <code>stop</code> ) | <code>ncat ozone.local 5050</code>      | <code>nc ozone.local 5050</code>                                       |

| TASK                                                  | WINWINDOWS — POWERSHELL        | MACMACOS — TERMINAL (ZSH)      |
|-------------------------------------------------------|--------------------------------|--------------------------------|
| Build & flash firmware                                | <code>pio run -t upload</code> | <code>pio run -t upload</code> |
| Erase flash<br>enter download mode, close PuTTY first | <code>pio run -t erase</code>  | <code>pio run -t erase</code>  |

Windows needs **Nmap** (for ncat) and **Apple Bonjour** to resolve `.local` names; macOS has `nc` and mDNS built in. The device connects out to `SECRET_PC_HOST:5000`, so the PC listener must be up *before* the run ends; the remote-stop server lives on the device at `:5050`. If `pio` isn't on PATH in PowerShell: &  
`"$env:USERPROFILE\.platformio\penv\Scripts\platformio.exe" run -t upload`.

OzoneTrckr · ESP32-S2 Feather + ADS131M04 · single rolling file `/lastrun.csv`, per-block commit, NTP clock, mDNS `ozone.local`, flicker-free TFT, latched servo pins on sleep. Working prototype — 2026-06-27.